kurvst Typst API

`kurvst` is a Typst package backed by `kurvst.wasm`, a small Kurbo-based plugin. Its public API is path-based: construct a path dictionary, pass that path into transforms, and draw the returned path.

It exposes Bezier utilities that are awkward or unavailable in native Typst:

- constructing cubic and Hobby paths,
- trimming paths by arc length,
- generating sampled path patterns,
- fitting Kurbo offset/parallel paths,
- converting returned path geometry to CeTZ drawing commands.

Path geometry points are two-item tuples:

```
#let p = (1.2, -0.4)
```

Cubic segments use `start`, `control-start`, `control-end`, and `end`:

```
#let segment = (
  start: (0, 0),
  control-start: (1, 0.5),
  control-end: (2, -0.5),
  end: (3, 0),
)
```

Use `from-cubic(segment)` once to turn that cubic dictionary into a path dictionary. Returned paths contain a curve-command wire path in `path`. Rust converts that wire path to Kurbo's `BezPath` internally, and every path-level Kurvst function accepts either a returned dictionary or the raw wire path.

```
#let path = kurvst.from-cubic(segment)
#let trimmed = kurvst.trim(path, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel(trimmed, distance: 0.15)
```

Path Wire Format

A Kurvst wire path is a dictionary with one `elements` array. The elements mirror Typst's native `curve` commands, but points are plain numeric tuples:

```
#let path = (
  elements: (
    (kind: "move", start: (0, 0)),
    (kind: "line", end: (0.6, 0.3)),
    (kind: "quad", control: (1.0, 1.0), end: (1.4, 0.3)),
    (
      kind: "cubic",
      control-start: (1.8, -0.4),
      control-end: (2.4, 1.0),
      end: (3.0, 0),
    ),
    (kind: "close", mode: "straight"),
  ),
)
```

The supported element kinds are:

- `move`: starts a subpath at `start`.
- `line`: adds a straight segment ending at `end`.
- `quad`: adds a quadratic segment using `control` and `end`.
- `cubic`: adds a cubic segment using `control-start`, `control-end`, and `end`.
- `close`: closes the current subpath. `mode` is optional and defaults to `"straight"` when emitted by Kurvst.

Prefer the path fragment helpers for new code. `line`, `quad`, and `cubic` include their start points, so fragments can be built independently:

```
#let custom = kurvst.path(
  kurvst.line((0, 0), (0.6, 0.3)),
  kurvst.quad((0.6, 0.3), (1.0, 1.0), (1.4, 0.3)),
```

```
  kurvst.cubic((1.4, 0.3), (1.8, -0.4), (2.4, 1.0), (3.0, 0)),
)
```

`path` and `append` flatten fragments. If an appended fragment starts where the current path ends, its leading `move` is skipped; if it starts elsewhere, the `move` begins a new subpath. The edited path can go straight back into Kurvst transforms:

```
#let base = kurvst.from-cubic(segment)
#let extended = kurvst.append(base, kurvst.line(segment.end, (3.4, 0.2)))

#let trimmed = kurvst.trim(extended, start-outset: 0.15)
#let shifted = kurvst.parallel(trimmed, distance: 0.1)

#kurvst.to-native(shifted, unit: 36pt, stroke: rgb("#1b7f4c") + 0.7pt)
```

Use `elements` when you need the raw command stream, `points` for the visited endpoints, `segments` for cubic segment dictionaries, `to-native` for native Typst drawing, and `to-cetz-data` or `to-cetz` for CeTZ.

A typical chain keeps the returned dictionaries intact until the final drawing step:

```
#let base = kurvst.hobby-through(demo-start, demo-through, demo-end)
#let trimmed = kurvst.trim(base, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel(trimmed, distance: 0.15)

#cetz.canvas({
  kurvst.to-cetz(base, stroke: rgb("#bbbbbb") + 0.4pt)
  kurvst.to-cetz(trimmed, stroke: rgb("#d72638") + 0.6pt)
  kurvst.to-cetz(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
})
```

Paths And Trimming

`cubic` constructs a path from one cubic Bezier. `trim` trims by curve distance from the start and/or end and returns another path dictionary.

```
#let path = kurvst.from-cubic(segment)
#let trimmed = kurvst.trim(path, start-outset: 0.15, end-outset: 0.1)
```

`hobby-through(start, through, end)` builds a smooth open curve through three points; `segments` returns the two cubic halves split at `through`. `hobby-spline(points)` does the same for any open point sequence with at least two points. Both return path dictionaries and can be passed directly to path-level helpers.

```
#let spline = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let shifted = kurvst.parallel(spline, distance: 0.14)
#let wiggle = kurvst.pattern(
  spline,
  pattern: kurvst.wave(samples-per-period: 24),
  amplitude: 0.1,
  wavelength: 0.55,
)
#cetz.canvas({
  kurvst.to-cetz(spline, stroke: rgb("#bbbbbb") + 0.45pt)
  kurvst.to-cetz(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
  kurvst.to-cetz(wiggle, stroke: rgb("#355c9a") + 0.55pt)
})
```

`split-through` combines Hobby spline construction with endpoint trimming and returns one path part between each consecutive point. Consumers such as Linnest can wrap it for graph-edge geometry.

Path Patterns

`pattern` generates a one-dimensional pattern along any path dictionary. Built-ins are regular Typst dictionaries:

```
#let wave = kurvst.wave()
#let zigzag = kurvst.zigzag()
#let coil = kurvst.coil(longitudinal-scale: 1.6)
```

String names `"wave"`, `"zigzag"`, and `"coil"` are accepted for convenience, but they are resolved in Typst before calling wasm.

Custom patterns use the same object shape:

```
#let hook = (
  kind: "points",
  name: "hook",
  interpolation: "linear",
  points: (
    (at: 0, x: 0, y: 0),
    (at: 0.25, x: 0.15, y: 1),
    (at: 0.5, x: 0, y: 0),
    (at: 0.75, x: -0.15, y: -1),
    (at: 1, x: 0, y: 0),
  ),
)
```

`at` is the phase position inside one wavelength, from `0` to `1`. The `x` coordinate offsets along the path tangent and `y` offsets along the path normal; both are scaled by `amplitude`. If `at` is omitted, points are spaced evenly. Use `interpolation: "linear"` for corners and `"smooth"` for a spline through sampled points.

`endpoint-ramp: true` tapers amplitude at anchored endpoints. This is useful for coils, whose longitudinal offset would otherwise put the first and last visible points inside the turn near nodes.

```
#let base = kurvst.from-cubic(segment)
#let path = kurvst.pattern(
  base,
  pattern: hook,
  amplitude: 0.15,
  wavelength: 0.7,
)
#native-scene({
  native-cubics(kurvst.segments(base), stroke: rgb("#c8c8c8") + 0.45pt)
  native-polyline(kurvst.points(path), stroke: rgb("#1b7f4c") + 0.8pt)
})
```

Parallel Paths

`parallel` uses Kurbo's offset curve fitter to produce a path at a fixed normal distance from the source path. Positive distances follow the left normal of the path direction; negative distances follow the right normal. `start-outset` and `end-outset` trim the fitted path by arc length after the offset is computed.

```
#let base = kurvst.from-cubic(segment)
#let left = kurvst.parallel(base, distance: 0.18, start-outset: 0.35, end-outset: 0.35)
#let right = kurvst.parallel(base, distance: -0.18)
#native-scene({
  native-cubics(kurvst.segments(base), stroke: rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(left), stroke: rgb("#1b7f4c") + 0.8pt)
  native-cubics(kurvst.segments(right), stroke: rgb("#355c9a") + 0.8pt)
})
```

Path Layers

`layer` is a convenience wrapper for drawing derived visible paths. It combines side-aware offsetting, endpoint outsets, and centered shortening into one path-in/path-out operation. `length` is a fixed visible arc length, `ratio` is a fraction of the input path length, and `resolve-length` decides how to combine them. The default `"min"` keeps whichever limit is shorter.

```
#let base = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
```

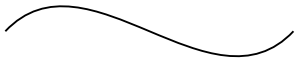
```
  (2.8, 0.4),
))
#let layer = kurvst.layer(
  base,
  offset: 0.16,
  length: 1.6,
  ratio: 0.5,
  resolve-length: "min",
)
#cetz.canvas({
  kurvst.to-cetz(base, stroke: rgb("#bbbbbb") + 0.45pt)
  kurvst.to-cetz(layer, stroke: rgb("#1b7f4c") + 0.8pt)
})
```

Use `side-point` to choose the sign of the offset from a point on the desired side of the path. Drawing packages can use this to place derived layers on the same side as an edge label without knowing anything about the physics or graph style that requested the layer.

Native Drawing Primitives

Kurvst returns dictionaries and arrays. The core geometry can be drawn without CeTZ by emitting native `curve` content:

```
#kurvst.to-native(kurvst.from-cubic(segment), unit:
36pt, stroke: black + 0.7pt)
```



The generated reference below keeps drawing code inline. It only uses the small common helpers above for coordinate scaling, native `curve` construction, point markers, and fixed-size preview blocks.

CeTZ Interoperability

The CeTZ helpers are thin adapters over the same returned geometry. Use them when the surrounding document already lives in a CeTZ canvas, or when you want CeTZ path merging and styling:

```
#cetz.canvas({
  let base = kurvst.from-cubic(segment)
  kurvst.to-cetz(base, stroke: rgb("#d72638") +
0.6pt)

  let path = kurvst.pattern(
    base,
    pattern: kurvst.coil(longitudinal-scale: 1.6),
    amplitude: 0.12,
    wavelength: 0.55,
  )
  kurvst.to-cetz(path, stroke: rgb("#355c9a") +
0.55pt)
})
```

Generated Reference

kurvst

- _length()
- _resolve-length-target()
- append()
- center-outset()
- close()
- coil()
- cubic()
- cubic-point()
- cubic-tangent()

- cubic-to()
- elements()
- from-cubic()
- from-elements()
- hobby-spline()
- hobby-through()
- layer()
- line()
- line-segment()
- line-to()
- move-to()
- parallel()
- path()
- pattern()
- point()
- points()
- quad()
- quad-to()
- segments()
- split-through()
- to-cetz()
- to-cetz-data()
- to-native()
- trim()
- wave()
- zigzag()

Variables
- layer-defaults

_length
Compute the arc length of a path dictionary.

Parameters
```
_length(
  path,
  accuracy
) -> int float
```

_resolve-length-target
Resolve a fixed and relative visible path length.

`length` is interpreted as a fixed arc length. `ratio` is interpreted as a fraction of `base-length`. `method` may be `"min"`/`"shorter"`, `"max"`/`"longer"`, `"length"`/`"fixed"`, `"ratio"`/`"relative"`, `"none"`/`"full"`, or a function receiving `(base-length, length, ratio)`.

Parameters
```
_resolve-length-target(
  base-length,
  length,
  ratio,
  method
) -> none int float
```

append

Return a path with additional fragments or elements appended.

If an appended fragment starts with a `move` at the current endpoint, the `move` is skipped. If it starts elsewhere, the `move` begins a new subpath.

Parameters

```
append(
  path,
  ..parts
) -> dictionary
```

center-outset

Compute the symmetric trim needed to center a shorter path layer.

Parameters

```
center-outset(
  base-length,
  length,
  ratio,
  resolve-length,
  start-outset,
  end-outset
) -> int float
```

close

Build a `close` path element.

Parameters

```
close(mode) -> dictionary
```

coil

A smooth coil path pattern.

```
#let pattern = kurvst.coil(samples-per-period:
24, longitudinal-scale: 1.6)
#let points = pattern.points.map(point => (
  point.at * 3.1 + point.x * 0.18,
  1 + point.y * 0.32,
))
#native-scene({
  native-polyline(((0.0, 1.0), (3.15, 1.0)),
stroke: rgb("#bbbbbb") + 0.35pt)
  native-polyline(points, stroke: rgb("#1b7f4c")
+ 0.75pt)
})
```

Parameters

```
coil(
  samples-per-period,
  longitudinal-scale
) -> dictionary
```

cubic

Build a cubic path fragment.

Parameters

```
cubic(
  start,
  control-start,
  control-end,
  end
) -> dictionary
```

cubic-point

Evaluate a cubic segment at parameter `t`.

Parameters

```
cubic-point(
  segment,
  t
) -> dictionary
```

cubic-tangent

Evaluate the tangent of a cubic segment at parameter `t`.

Parameters

```
cubic-tangent(
  segment,
  t
) -> dictionary
```

cubic-to

Build a `cubic` path element.

Parameters

```
cubic-to(
  control-start,
  control-end,
  end
) -> dictionary
```

elements

Return the command elements that make up a Kurvst path.

The elements mirror Typst's native `curve` components with numeric point tuples: `move`, `line`, `quad`, `cubic`, and `close`.

Parameters

```
elements(path) -> array
```

from-cubic

Build a path fragment from a cubic segment dictionary.

Parameters

```
from-cubic(segment) -> dictionary
```

from-elements

Build a path dictionary from an existing element array.

Parameters

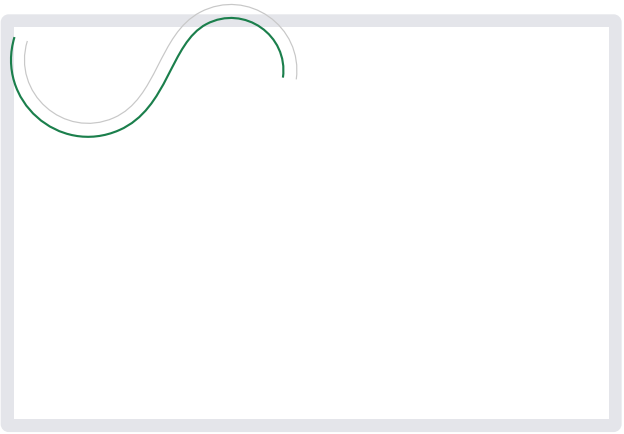
```
from-elements(elements) -> dictionary
```

hobby-spline

Build an open Hobby spline through two or more points.

The returned dictionary is a path dictionary backed by a curve-command wire path. Rust converts that wire path to Kurbo's `BezPath` internally, so it can be passed directly to `parallel()`, `pattern()`, `trim()`, or `to-native()`.

```
#let spline = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let shifted = kurvst.parallel(spline, distance:
0.14)
#native-scene({
  native-cubics(kurvst.segments(spline), stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(shifted),
stroke: rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

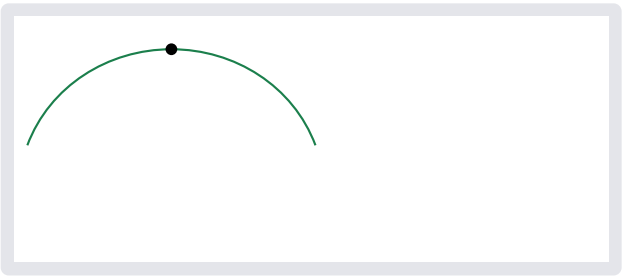
```
hobby-spline(
  points,
  omega,
  accuracy
) -> dictionary
```

hobby-through

Build an open Hobby curve through `start`, `through`, and `end`.

The returned dictionary is a path dictionary. For this three-point variant, `segments()` returns the two cubic halves meeting at `through`.

```
#let path = kurvst.hobby-through(demo-start,
demo-through, demo-end, omega: 1.2)
#native-scene({
  native-cubics(kurvst.segments(path), stroke:
rgb("#1b7f4c") + 0.8pt)
  native-dot(demo-through, fill: black)
})
```



Parameters

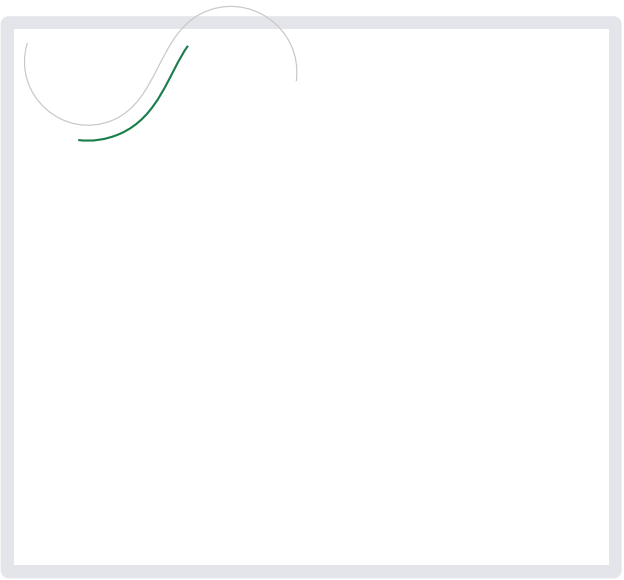
```
hobby-through(
  start,
  through,
  end,
  omega,
  accuracy
) -> dictionary
```

layer

Build a derived visible path layer.

`layer` combines the common operations needed by drawing packages: optional side-aware offsetting, endpoint trimming, and centered shortening by a fixed `length`, a relative `ratio`, or both. The return value is a normal Kurvst path dictionary that can be passed to `pattern()`, `parallel()`, `trim()`, or `to-cetz()`.

```
#let base = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let layer = kurvst.layer(
  base,
  offset: 0.16,
  length: 1.6,
  ratio: 0.5,
  resolve-length: "min",
)
#native-scene({
  native-cubics(kurvst.segments(base), stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(layer), stroke:
rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

```
layer(
  path,
  offset,
  length,
  ratio,
  resolve-length,
  start-outset,
  end-outset,
  side-point,
  accuracy,
  optimize
) -> dictionary
```

line

Build a straight-line path fragment.

Parameters

```
line(
  start,
  end
) -> dictionary
```

line-segment

Build a cubic segment dictionary for a straight line.

Parameters

```
line-segment(
  start,
  end
) -> dictionary
```

line-to

Build a `line` path element.

Parameters

```
line-to(end) -> dictionary
```

move-to

Build a `move` path element.

Parameters

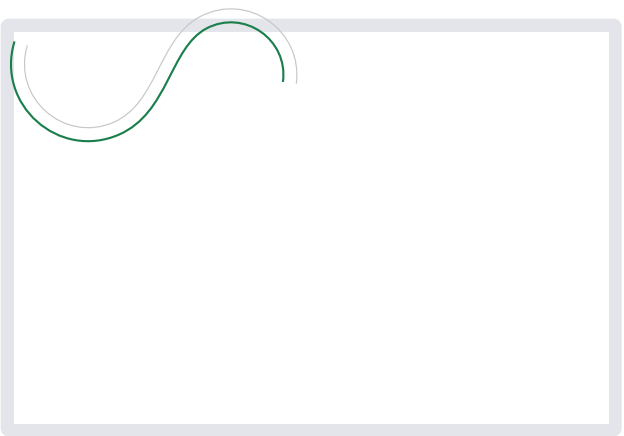
```
move-to(start) -> dictionary
```

parallel

Generate a parallel path for a path.

Pass any path returned by this module, such as `hobby-spline()`. The result is a drawable path dictionary with the fitted parallel path and its curve-command wire path.

```
#let spline = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let parallel = kurvst.parallel(spline,
distance: 0.14)
#native-scene({
  native-cubics(kurvst.segments(spline), stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(parallel),
stroke: rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

```
parallel(
  path,
  distance,
  start-outset,
  end-outset,
  accuracy,
  optimize
) -> dictionary
```

path

Build a path dictionary from path fragments or elements.

Leading `move` elements are kept. Later `move` elements are skipped when they match the current endpoint, so `path(line(a, b), line(b, c))` produces one continuous subpath.

Parameters

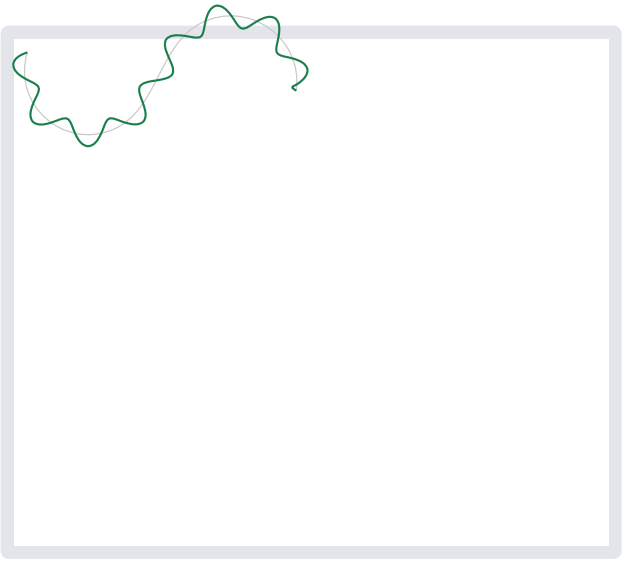
```
path(..parts) -> dictionary
```

pattern

Generate a sampled 1D pattern along a path.

`pattern` may be `wave()`, `zigzag()`, `coil()`, a compatible string name, or a point pattern: `(kind: "points", interpolation: "linear" or "smooth", points: ((at: 0, x: 0, y: 0), ...))`. The whole input path is sampled continuously, so pattern phase does not restart at cubic segment boundaries.

```
#let spline = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let path = kurvst.pattern(
  spline,
  pattern: kurvst.wave(samples-per-period: 24),
  amplitude: 0.12,
  wavelength: 0.55,
)
#native-scene({
  native-cubics(kurvst.segments(spline), stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(path), stroke:
rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

```
pattern(
  path,
  pattern,
  amplitude,
  wavelength,
  phase,
  samples-per-period,
  coil-longitudinal-scale,
  anchor-start,
  anchor-end,
  accuracy
) -> dictionary
```

point

Build a numeric point tuple.

Parameters

```
point(
  x,
  y
) -> array
```

points

Return the points visited by a Kurvst path's command stream.

Parameters

```
points(path) -> array
```

quad

Build a quadratic path fragment.

Parameters

```
quad(
  start,
  control,
  end
) -> dictionary
```

quad-to

Build a `quad` path element.

Parameters

```
quad-to(
  control,
  end
) -> dictionary
```

segments

Return drawable cubic segments for any Kurvst path dictionary.

Lines and quadratics are converted to equivalent cubic segments so downstream renderers can use one code path.

Parameters

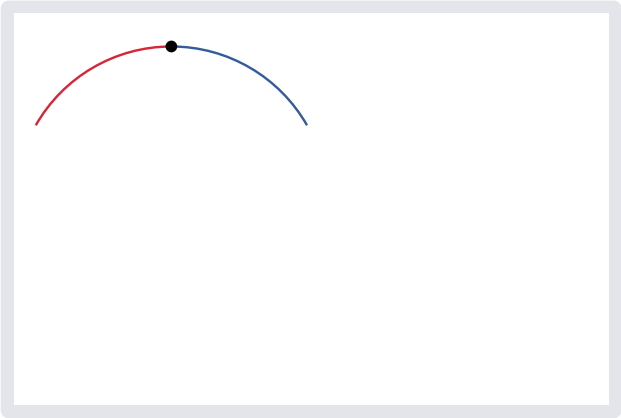
```
segments(path) -> array
```

split-through

Split a path through a point sequence into per-span paths.

The returned dictionary has `parts` and `curve`. Each entry in `parts` is the path between two consecutive points, and all parts join smoothly through the input points.

```
#let split = kurvst.split-through(
  (demo-start, demo-through, demo-end),
  start-outset: 0.2,
  end-outset: 0.2,
)
#native-scene({
  native-
cubics(kurvst.segments(split.parts.at(0)),
stroke: rgb("#d72638") + 0.9pt)
  native-
cubics(kurvst.segments(split.parts.at(1)),
stroke: rgb("#355c9a") + 0.9pt)
  native-dot(demo-through, fill: black)
})
```



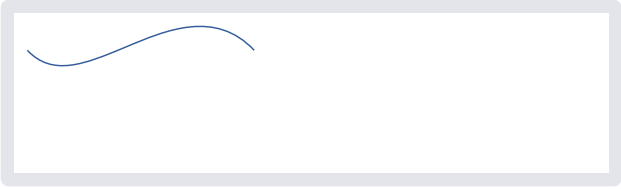
Parameters

```
split-through(
  points,
  omega,
  start-outset,
  end-outset,
  accuracy
) -> dictionary
```

to-cetz

Draw a path dictionary through CeTZ.

```
#let path = kurvst.parallel(kurvst.from-
cubic(demo-segment), distance: 0.18)
#cetz.canvas({
  kurvst.to-cetz(path, stroke: rgb("#355c9a") +
0.55pt)
})
```



Parameters

```
to-cetz(
  path,
  unit,
  ..style
) -> content
```

to-cetz-data

Emit a Kurvst path as CeTZ path data.

The result has CeTZ's subpath shape: `(origin, closed, segments)`, where line segments are `("l", end)` and cubics are `("c", control-start, control-end, end)`.

Parameters

```
to-cetz-data(
  path,
  unit
) -> array
```

to-native

Emit a Kurvst path as native Typst `curve` content.

Parameters

```
to-native(
  path,
  unit,
  ..style
) -> content
```

trim

Trim a path by curve distance from either end.

The returned dictionary is another path dictionary with a curve-command wire path.

```
#let base = kurvst.from-cubic(demo-segment)
#let trimmed = kurvst.trim(base, start-outset:
0.35, end-outset: 0.3)
#native-scene({
  native-cubics(kurvst.segments(base), stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(trimmed),
stroke: rgb("#d72638") + 1pt)
})
```

Parameters

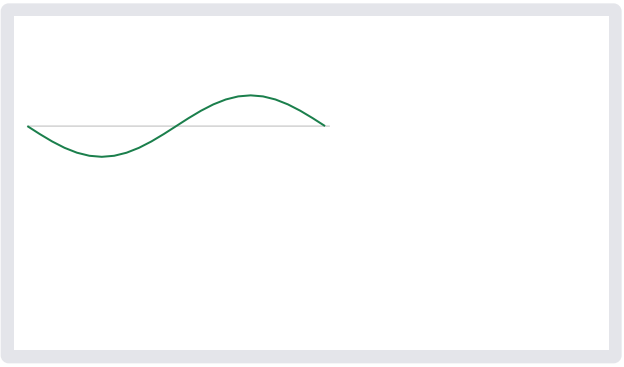
```
trim(
  path,
  start-outset,
  end-outset,
  accuracy
) -> dictionary
```

wave

A smooth sinusoidal path pattern.

```
#let pattern = kurvst.wave(samples-per-period:
24)
#let points = pattern.points.map(point => (
  point.at * 3.1 + point.x * 0.18,
```

```
  1 + point.y * 0.32,
))
#native-scene({
  native-polyline(((0.0, 1.0), (3.15, 1.0)),
stroke: rgb("#bbbbbb") + 0.35pt)
  native-polyline(points, stroke: rgb("#1b7f4c")
+ 0.75pt)
})
```



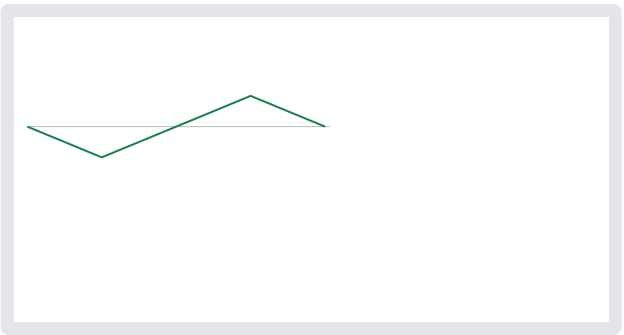
Parameters

`wave(samples-per-period) -> dictionary`

zigzag

A straight-segment triangular path pattern.

```
#let pattern = kurvst.zigzag()
#let points = pattern.points.map(point => (
  point.at * 3.1 + point.x * 0.18,
  1 + point.y * 0.32,
))
#native-scene({
  native-polyline(((0.0, 1.0), (3.15, 1.0)),
stroke: rgb("#bbbbbb") + 0.35pt)
  native-polyline(points, stroke: rgb("#1b7f4c")
+ 0.75pt)
})
```



Parameters

`zigzag() -> dictionary`

layer-defaults `dictionary`

Default geometry options for derived path layers.

These defaults are shared by `layer()` and drawing packages that build on Kurvst. The dictionary shape intentionally mirrors the public function arguments so callers can merge user styles into it and unpack the result.